



VEHICLE SERVICE MANAGEMENT SYSTEM

By : M.Sharanya

Email : sharanyamaddha@gmail.com

PROJECT OVERVIEW

Automobile service centers rely on manual or semi-automated processes, making it difficult to track service history, schedule technicians efficiently, manage spare parts, and monitor workload and revenue. This lack of a centralized system results in delays, poor visibility, and reduced customer satisfaction, creating the need for a secure and scalable Vehicle Service Management System.

Solution :

- Centralized platform to manage vehicle service requests and complete service history.
- Automated technician and service bay assignment based on workload and specialization.
- Real-time inventory and spare parts tracking with low-stock alerts.
- End-to-end service lifecycle tracking from booking to closure.
- Role-based dashboards for Admin, Manager, Technician, and Customer.
- Automated billing, invoicing, and email notifications for better customer experience.

MICROSERVICES ARCHITECTURE

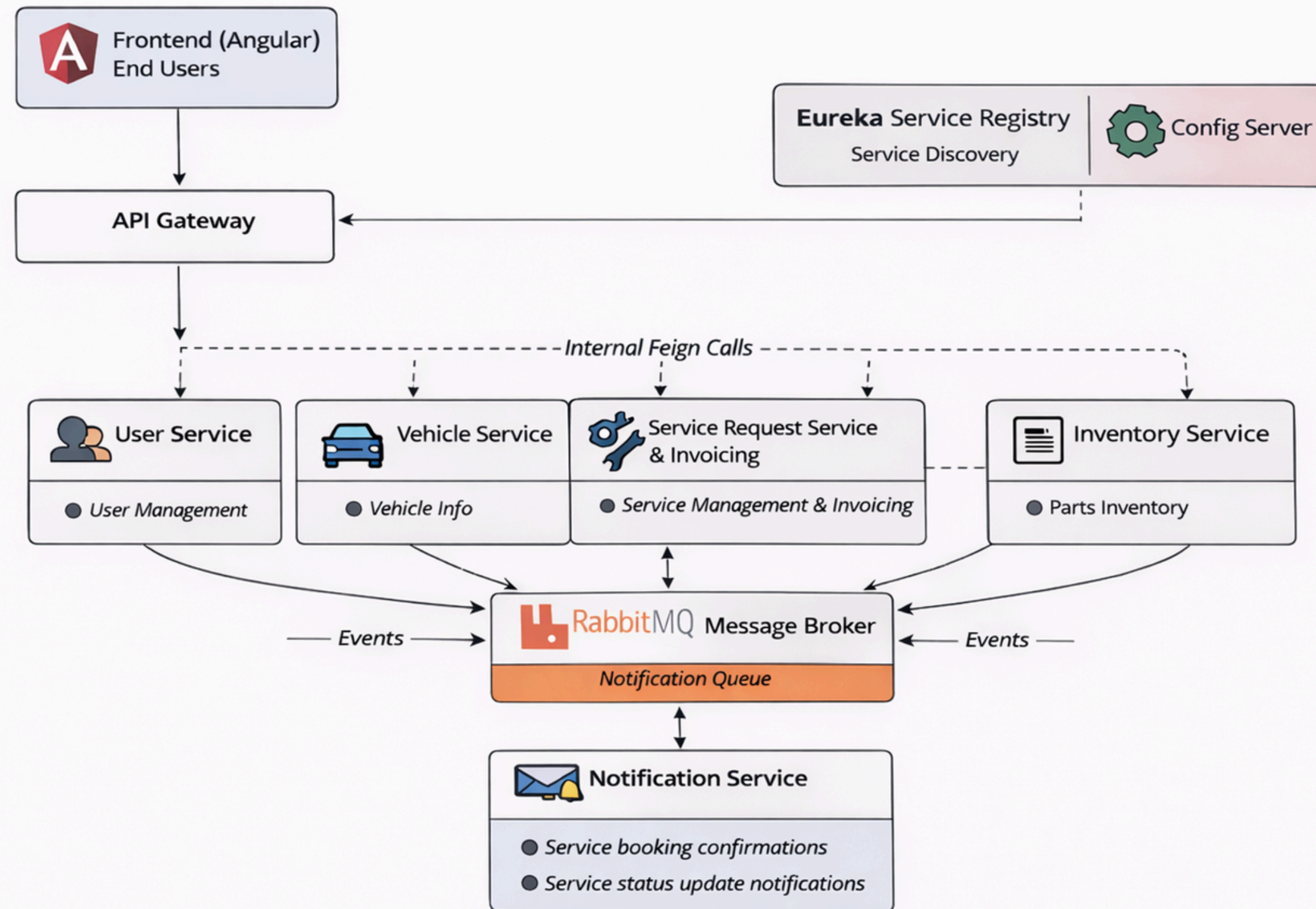
8 Microservices:

1. Config Server (8888) - Centralized configuration
2. Eureka Server (8761) - Service discovery
3. API Gateway (8080) - Entry point, JWT authentication, routing
4. User Service - Authentication, user management
5. Vehicle Service - Vehicle registration
6. Inventory Service - Spare parts, part requests
7. Service Request Service - Service lifecycle, billing, bays
8. Notification Service - Email notifications

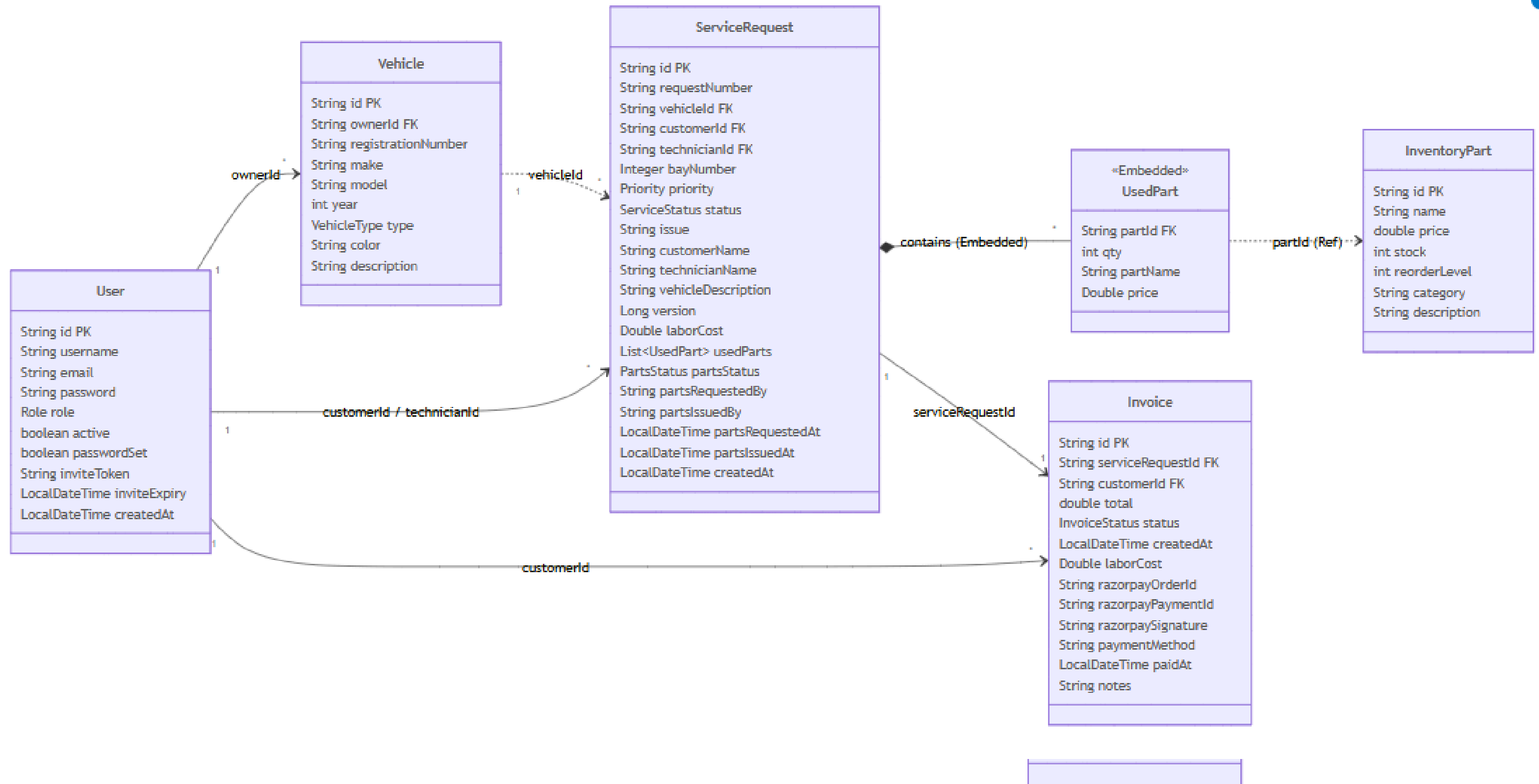
Inter-Service Communication:

- Feign Clients
- RabbitMQ

MICROSERVICES ARCHITECTURE



DATABASE SCHEMA



USER ROLES & RESPONSIBILITIES

1. Admin :

- Approves managers and technicians, manages user accounts.
- Controls system settings, pricing, and overall monitoring.

2. Manager :

- Assigns service requests to technicians and service bays.
- Tracks workload, availability, and service progress.

3. Technician :

- Performs assigned services and updates job status.
- Adds remarks and marks services as completed.

4. Customer :

- Books vehicle services and tracks service status.
- Views service history and invoices.

SERVICE REQUEST WORKFLOW

1.Book Service (Customer)

- Creates request (eg : SR-1234) → Status: **REQUESTED**

2.Assign Job (Manager)

- Selects Technician & Bay → Status: **ASSIGNED**

3.Execution & Parts (Technician)

- Starts Job → Status: **IN_PROGRESS**
- Requests Parts → Manager Approves → *PARTS_APPROVED*

4.Completion (Technician)

- Finishes work → Status: **COMPLETED**

5.Close & Bill (Manager)

- Finalizes labor cost.
- System Actions: Deducts Inventory Stock + Generates Invoice.
- Status: **CLOSED**

6.Payment (Customer)

- Pays via Razorpay → Invoice Status: *PAID*

KEY FEATURES IMPLEMENTED

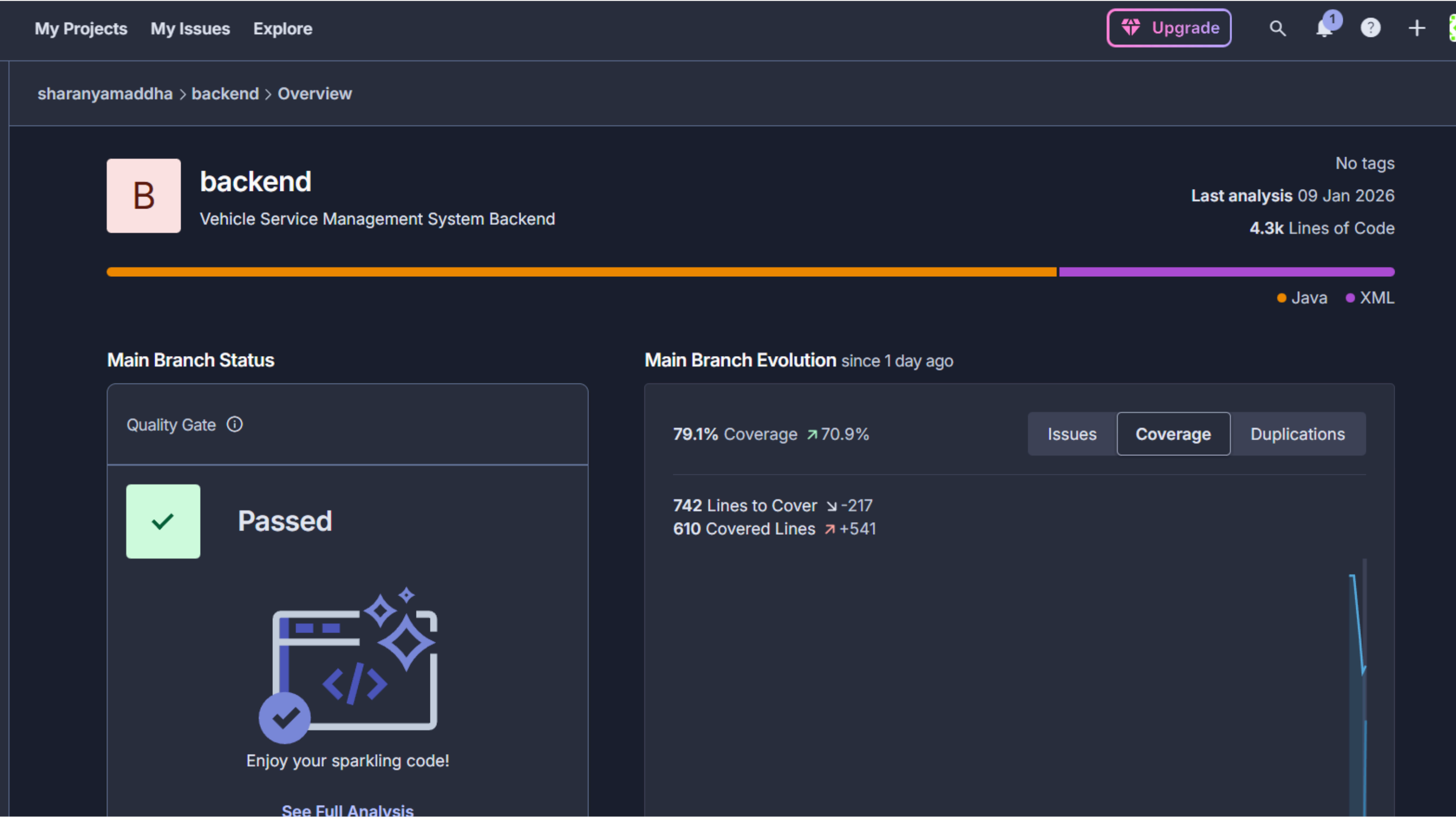
Feature Module	Functionality	How It Works (Implementation)
Onboarding	Secure Account Activation	Admins create accounts which stay Inactive until the user clicks a unique, time-sensitive Invitation Link (Token-based) to set their password.
Smart Assignment	Workload Balancing	System automatically blocks assignment if a Technician has reached their daily job capacity (e.g., 5 jobs/day).
Resource Management	Bay Conflict Resolution	Real-time validation prevents assigning multiple vehicles to the same Service Bay simultaneously.
Inventory Control	2-Step Parts Verification	Technicians must Request parts, but stock is only deducted after a Manager Approves them, preventing theft/waste.
Payments	Razorpay Integration	Secure payment gateway with Signature Verification to ensure transaction authenticity before updating invoice status.
Analytics	Technician Statistics	Real-time dashboards showing Active Workload (Current Jobs) vs. Performance History (Completed Jobs) for Manager oversight.
Resilience	Zero-Downtime Alerts	Assigned jobs proceed even if Inventory Service is down (Circuit Breaker), syncing stock data later.

CI/CD PIPELINE

 **Jenkins** / VehicleManagementSystem ▾ / #22 ▾ / Console Output

```
Container rabbitmq Waiting
Container eureka-server Waiting
Container config-server Waiting
Container notification-service Started
Container inventory-service Started
Container api-gateway Started
Container mongo Healthy
Container config-server Healthy
Container eureka-server Healthy
Container rabbitmq Healthy
Container service-request-service Starting
Container service-request-service Started
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline completed successfully!
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

SONARQUBE



DOCKER

 **docker desktop** PERSONAL

Ctrl+K

?

13

S

-

Ask Gordon BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit BETA

Docker Hub

Docker Scout

Extensions

<

 **backend**
D:\CHUBB CAPSTONE\backend

View configurations

 **rabbitmq** ●
[rabbitmq:ma](#)
[15672:15672](#)
[Show all por](#)

 **mongo** ●
[mongo:lates](#)

 **config-s...** ●
[backend-con](#)
[8888:8888](#)

 **eureka-...** ●
[backend-eur](#)

 **service-...** ●
[backend-ser](#)

 **api-gate...** ●
[backend-api-](#)

 **notifica...** ●
[backend-noti](#)

 **user-se...** ●
[backend-use](#)

 **inventor...** ●
[backend-inve](#)

 **vehicle-...** ●
[backend-veh](#)

config-server

```
o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-01-07T00:13:47.132Z INFO 1 --- [eureka-server] [      main]
o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache
Tomcat/11.0.15]
2026-01-07T00:13:47.225Z INFO 1 --- [eureka-server] [      main]
b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization
completed in 6053 ms

2026-01-07T00:13:48.195Z INFO 1 --- [config-server] [reshExecutor-%d]
com.netflix.discovery.DiscoveryClient : Disable delta property : false
2026-01-07T00:13:48.196Z INFO 1 --- [config-server] [reshExecutor-%d]
com.netflix.discovery.DiscoveryClient : Single vip registry refresh property :
null
2026-01-07T00:13:48.198Z INFO 1 --- [config-server] [reshExecutor-%d]
com.netflix.discovery.DiscoveryClient : Force full registry fetch : false
2026-01-07T00:13:48.198Z INFO 1 --- [config-server] [reshExecutor-%d]
com.netflix.discovery.DiscoveryClient : Application is null : false
2026-01-07T00:13:48.198Z INFO 1 --- [config-server] [reshExecutor-%d]
com.netflix.discovery.DiscoveryClient : Registered Applications size is zero :
true
2026-01-07T00:13:48.198Z INFO 1 --- [config-server] [reshExecutor-%d]
com.netflix.discovery.DiscoveryClient : Application version is -1: true
2026-01-07T00:13:48.199Z INFO 1 --- [config-server] [reshExecutor-%d]
com.netflix.discovery.DiscoveryClient : Getting all instance registry info from
the eureka server
2026-01-07T00:13:48.298Z INFO 1 --- [config-server] [foReplicator-%d]
com.netflix.discovery.DiscoveryClient : DiscoveryClient_CONFIG-
SERVER/fceb67f58799:config-server:8888: registering service...
2026-01-07T00:13:50.370Z INFO 1 --- [config-server] [nio-8888-exec-6]
o.s.c.c.s.e.NativeEnvironmentRepository : Adding property source: Config resource
'file [/tmp/config-repo-12076559325260740504/notification-service-docker.properties]'
via location 'file:/tmp/config-repo-12076559325260740504/'
```

Engine running

RAM 5.93 GB CPU 97.22% Disk: 27.46 GB used (limit 1006.85 GB)

>_ Terminal

Update available

DOCKER

docker

desktop

PERSONAL

Search

Ctrl+K

13

S

Ask Gordon

BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit

BETA

Docker Hub

Docker Scout

Extensions

Images

Give feedback

Local

My Hub

3.97 GB / 8.8 GB in use

35 images

Last refresh: 36 minutes ago

Search

	Name	Tag	Image ID	Created	Size	Actions
☐	mongo	latest	7f5bbdafebde	18 days ago	1.25 GB	
☐	rabbitmq	management-alpine	f10f014d87ff	20 days ago	273.54 MB	
☐	mysql	8.0	0275a35e79c6	1 month ago	1.07 GB	
☐	mongo	6	0a83b2f824b2	3 months ago	1.05 GB	
☐	confluentinc/cp-kafka	7.5.0	fbbb6fa11b25	2 years ago	1.33 GB	
☐	confluentinc/cp-zookeeper	7.5.0	02f6c042bb9a	2 years ago	1.33 GB	
☐	backend-service-request-service	latest	3d51630dcee7	32 minutes ag	880.98 MB	
☐	backend-notification-service	latest	6de7160036a2	32 minutes ag	882.2 MB	
☐	backend-api-gateway	latest	76c8dfcb8e94	33 minutes ag	845.67 MB	
☐	backend-inventory-service	latest	6f38eef09586	33 minutes ag	858.4 MB	

Showing 35 items

Engine running

RAM 6.37 GB CPU 0.75% Disk: 27.47 GB used (limit 1006.85 GB)

Terminal

Update available

DOCKER

 **docker desktop** PERSONAL

Search

Ctrl+K

S

—





 Ask Gordon BETA

 Containers

 Images

 Volumes

 Kubernetes

 Builds

 Models

 MCP Toolkit BETA

 Docker Hub

 Docker Scout

 Extensions

Images [Give feedback](#)

Local

My Hub

3.97 GB / 8.8 GB in use

35 images

Search

☐	Name	Tag	Image ID	Created	Size	Actions
☐	 mongo	o	0a83d21824d2	3 months ago	1.05 GB	   
☐	 confluentinc/cp-kafka 	7.5.0 	fbbb6fa11b25 	2 years ago	1.33 GB	   
☐	 confluentinc/cp-zookeeper	7.5.0	02f6c042bb9a	2 years ago	1.33 GB	   
☐	 backend-service-request-service	latest	3d51630dcee7	33 minutes ag	880.98 MB	   
☐	 backend-notification-service	latest	6de7160036a2	33 minutes ag	882.2 MB	   
☐	 backend-api-gateway 	latest 	76c8dfcb8e94 	33 minutes ag	845.67 MB	   
☐	 backend-inventory-service	latest	6f38eef09586	33 minutes ag	858.4 MB	   
☐	 backend-config-server	latest	79b362f69500	33 minutes ag	857.74 MB	   
☐	 backend-eureka-server	latest	8d9215cc0609	33 minutes ag	866.92 MB	   
☐	 backend-user-service	latest	c8dc0bdf3363	33 minutes ag	875.42 MB	   
☐	 backend-vehicle-service	latest	697b894cc0cd	33 minutes ag	858.39 MB	   

Showing 35 items

 Engine running

||



RAM 6.39 GB CPU 0.50% Disk: 27.47 GB used (limit 1006.85 GB)

 Terminal

 Update available

SUMMARY

- Microservices Architecture with 8 independent services
- Secure JWT-based Authentication & Role-Based Access Control .
- Secure Account Activation (Token-based) & Razorpay Payment Integration.
- Complete Service Lifecycle Management (Pending → Assigned → In Progress → Closed).
- Smart Workload Balancing & Real-time Bay Conflict Resolution.
- Spare Parts Inventory Management with 2-Step Verification (Request → Approve).
- Real-time Email Notifications via RabbitMQ
- Automated Billing & Dynamic Invoice Generation.
- CI/CD Pipeline with Jenkins, Docker & SonarCloud.



THANK YOU